

ISC Student Cluster Competition 2026: ASCOT5

Miro Palmu

University of Helsinki

23-02-2026



About me

- PhD student at University of Helsinki.
- Working on GPU port of astrophysical plasma HPC code called Runko.
- Part of 2023 team which placed 2nd at IndySCC.



- ASCOT5 is a test **particle orbit-following** code for toroidal magnetically confined fusion devices
- The code uses the **Monte Carlo method** to solve the distribution of particles by following their trajectories.

- The **evolution of the distribution function** for a test particle species a is described by the **Fokker-Planck equation**

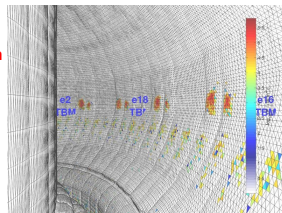
$$\frac{\partial f_a}{\partial t} + \mathbf{v} \cdot \nabla f_a + \frac{q_a}{m_a} (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{v}} f_a = \sum_b -\nabla_{\mathbf{v}} \cdot [\mathbf{a}_{ab} f_a - \nabla_{\mathbf{v}} \cdot (\mathbf{D}_{ab} f_a)]$$

and **approximated by the Langevin equation** for a large number of markers that represent the distributed function:

$$d\mathbf{z} = [\dot{\mathbf{z}} + \mathbf{a}(\mathbf{z}, t)] dt + \boldsymbol{\sigma}(\mathbf{z}, t) \cdot d\mathcal{W}$$

- The particles undergo **collisions with a static Maxwellian background plasma**
- The detailed magnetic fields and the first wall can be **fully 3D**
- MPI + OpenMP** (task-based) and **highly vectorized**

SCITAS

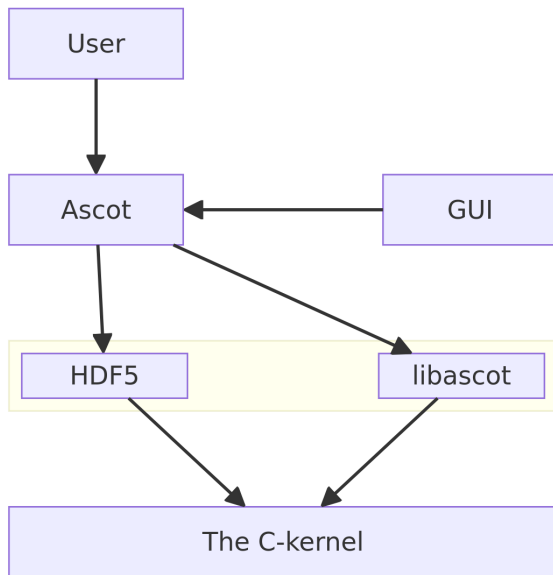


https://hpcfusion.bsc.es/2024/wp-content/uploads/sites/8/2024/11/Mathieu-Peybernes_FusionHPC-Workshop-1.pdf

ASCOT5 details

- ASCOT5 slides at SCC github.
- ASCOT5 docs:
<https://ascot4fusion.github.io/ascot5/index.html>
- TOKMAK fusion reactor:
<https://en.wikipedia.org/wiki/Tokamak>

ASCOT5 structure



What is HDF5?

- Hierarchical Data Format version 5 (HDF5) is a widely used, open-source, **binary file format** and data model designed for storing and managing large and complex datasets.
- It supports a hierarchical structure (similar to a filesystem):
 - ▶ Groups (like folders)
 - ▶ Datasets (like files containing arrays of data)
- Allows self-describing metadata, meaning the data carries its own structure and description.

Why binary file format?

- **Performance:** Binary files avoid the overhead of parsing text, delivering much faster I/O speeds.
- **Compactness:** Numerical data stored in binary takes significantly less space than ASCII text.
- **Precision Preservation:** Stores floating-point and integer data exactly without rounding or formatting issues.
- **Scalability:** HPC simulations often generate terabytes of data – binary is required to handle such volumes efficiently.
- **Parallel I/O Support:** Text formats typically cannot support safe and efficient parallel writes from thousands of processes.

HDF5 files

HDF5 files are handled using the HDF5 libraries or with CLI tools:

h5ls: List Contents

```
h5ls mydata.h5  
h5ls -r mydata.h5    # recursive listing
```

h5dump: Detailed Inspection

```
h5dump mydata.h5  
h5dump -H mydata.h5    # header only (no raw data)
```

HDF5 files

h5stat: File Statistics

```
h5dump mydata.h5
```

h5check: File Integrity Check

```
h5check mydata.h5
```

GUI tools: **HDFView**, **ParaView**, **VisIt**, etc.

Basic structure of ASCOT5 simulation

- Generate `ascot.h5` input data using ASCOT5 python API
 - ▶ This includes generating the markers.
- Run `ascot5_main --d="<description>"` in the directory where `ascot.h5` is.
 - ▶ There can be multiple input files in the same file, but only one can be active.
 - ▶ ASCOT5 will write the output to the same `ascot.h5` file.
 - ▶ Separate outputfile can be specified manually, but it is recommended to not do this.
- Post-process the output data using ASCOT5 python API.

HDF5 in ASCOT5

- All ASCOT5 inputs and results are contained in a single HDF5 file.
- Data in the HDF5 should **not** be directly accessed and the Python API should be used instead.
- Each input and each result has a ten number quasi-unique identifier (QID) randomly assigned when the data is written.
- Each result group stores QIDs of the inputs that were used in the simulation.

Compiling ASCOT5

`https://ascot4fusion.github.io/ascot5/installing.html`

GPU accelerated ASCOT5 on Mahti

```
# nvhpc-hpcx-cuda12 version is important.
# With other versions there might be mismatch in which hdf5 headers are used
# vs. which hdf5 shared library is loaded during runtime.
#
# Mahti documentation (docs.csc.fi/computing/compiling-mahti/)
# recommends nvhpc-hpcx-cuda12/25.1 but it did not work.
#
# Puhti documentation (docs.csc.fi/computing/compiling-puhti/)
# recommends nvhpc-hpcx-cuda12/24.11 which worked.
module purge
module use /appl/opt/nvhpc/modulefiles
module load nvhpc-hpcx-cuda12/24.11
module load gcc/11.2.0 openmpi/4.1.2-cuda # dependencies of hdf5/1.10.7-mpi
module load hdf5/1.10.7-mpi
module load hpcx # At runtime fixes undefined symbol: opal_libevent2022_evthread_use_pthreads
module load python-data/3.12-25.09

python -m venv ascot-env
source ascot-env/bin/activate

git clone https://github.com/ascot4fusion/ascot5.git
cd ascot5

pip install -e .
pip install matplotlib # Required for ASCOT5 python module.

make ascot5_main -j 8 MPI=1 ACC=1 CC='nvc -lmpi'
make libascot -j 8 MPI=1 ACC=1 CC='nvc -lmpi'
```

Optimizations

- Compilation flags.
- There is no dynamic load balancing.
 - ▶ Some MPI ranks might run out of markers while others still have work to do.
 - ▶ It is "advisable to randomly permute the markers before writing them to disk or initializing them to ensure a more balanced distribution of work."
- <https://ascot4fusion.github.io/ascot5/optimization.html>